

Reliability: Consistency, Availability, CAP Theorem, Fault Tolerance, and Graceful Degradation

1. Overview

Reliability in system design means a system continues to behave correctly and usefully even when machines fail, networks slow down, databases lag, or traffic spikes.

Definition

Reliability is the ability of a system to provide correct service over time despite failures.

In distributed systems, reliability is usually discussed through these ideas:

Concept	Simple Meaning
Consistency	Users see correct and expected data
Availability	The system responds to requests
CAP theorem	During network partition, choose consistency or availability
Fault tolerance	The system survives failures
Graceful degradation	The system keeps core features working when some parts fail

Why It Matters

Modern systems run on many machines. Any machine, disk, database, cache, queue, service, or network link can fail. A reliable system does not assume perfect conditions.

Reliability matters because:

- Users expect systems to work continuously.
- Businesses lose money during downtime.
- Data corruption can be worse than temporary unavailability.
- Large systems fail partially, not always completely.
- Interviewers want to see if you understand real-world trade-offs.

Where It Is Used in Real Systems

Reliability is used in:

- Payment systems
- Banking applications
- Social media feeds
- Search engines
- Ride-hailing systems
- E-commerce order placement
- Distributed databases

- Messaging systems
- Cloud services
- Backend microservices

Why Interviewers Ask About It

Interviewers ask about reliability because system design is mostly about trade-offs. They want to know whether you can decide:

- Should the system reject a request or return stale data?
- Should writes be synchronous or asynchronous?
- Should the database be strongly consistent or eventually consistent?
- What happens when one region goes down?
- What happens if cache, queue, or payment service fails?

Good candidates do not say "make it scalable and reliable" vaguely. They explain failure modes and trade-offs clearly.

2. Core Idea

The core idea is: distributed systems cannot guarantee everything perfectly all the time. You design for expected failures and choose the right trade-offs for the business requirement.

Intuition

Imagine an application with three servers and one database. If one server crashes, users should still be served by the other two servers. If the database replica is behind, the system must decide whether to show slightly old data or wait for the latest data.

Reliability is about making these decisions intentionally.

Real-World Analogy

Think of a restaurant:

- Consistency: The waiter gives the correct order to the customer.
- Availability: The restaurant continues serving customers.
- Fault tolerance: If one cook is absent, another cook handles the order.
- Graceful degradation: If dessert is unavailable, the restaurant still serves main courses.
- CAP theorem: If the kitchen and billing counter cannot communicate, should they stop taking orders or continue with possible mismatch?

Small Example

Suppose a user updates their profile name from **Rahul** to **Rahul Sharma**.

With strong consistency:

1. User sends update request.
2. System writes to primary database.
3. Replicas confirm the update.

4. User and other users see **Rahul Sharma**.

With eventual consistency:

1. User sends update request.
2. Primary database updates immediately.
3. Replicas update asynchronously.
4. Some users may briefly see **Rahul**.
5. After replication finishes, everyone sees **Rahul Sharma**.

Both designs can be correct depending on the system.

Step-by-Step Explanation

1. A system receives reads and writes.
2. It stores data across machines for scale and fault tolerance.
3. Machines communicate over a network.
4. Networks can fail or become slow.
5. During failure, the system must choose whether to:
 - wait for latest correct data, or
 - respond quickly with possibly stale data, or
 - reject the request safely.
6. That choice defines the system's reliability behavior.

3. Important Subtopics

3.1 Consistency

What It Means

Consistency means all users and services see correct data according to the system's rules.

In distributed systems, consistency usually means whether all replicas show the same data at the same time.

Why It Matters

Wrong data can cause serious issues:

- Double payment
- Overselling inventory
- Wrong bank balance
- Duplicate booking
- Incorrect permissions

Example

In a banking system, after transferring Rs. 1000 from account A to account B, the total money should remain correct. The debit and credit should not be partially visible.

Common Interview Angle

Interviewers may ask whether your design needs strong consistency or eventual consistency.

Example answer:

For payments and inventory updates, I prefer strong consistency. For feeds, likes, view counts, and analytics, eventual consistency is usually acceptable.

3.2 Availability

What It Means

Availability means the system responds to every valid request, even if some components are failing.

Availability does not always mean the response contains the latest data. It means the system gives a non-error response within an acceptable time.

Why It Matters

Users care about whether the system works. A system with perfect data but frequent downtime may be unacceptable.

Example

A social media feed may continue showing cached posts even if the recommendation service is down.

Common Interview Angle

Interviewers may ask how to keep a system available during database, cache, or region failure.

Common answer points:

- Replication
- Load balancing
- Health checks
- Failover
- Caching
- Circuit breakers
- Read replicas
- Multi-region deployment

3.3 CAP Theorem

What It Means

CAP theorem says that in a distributed system, when a network partition happens, the system must choose between:

- Consistency: return only the latest correct data
- Availability: return a response even if data may be stale

Partition tolerance is not optional in real distributed systems because networks can fail.

Why It Matters

CAP helps you reason about behavior during network failures.

Example

Two database replicas cannot communicate.

If both accept writes, the system remains available but may become inconsistent.

If one side rejects writes, the system protects consistency but reduces availability.

Common Interview Angle

Interviewers often ask:

Is this system CP or AP?

Better answer:

During partition, this design favors consistency for writes because incorrect orders are worse than temporary rejection. Reads may still use cached data where safe.

3.4 Fault Tolerance

What It Means

Fault tolerance means the system keeps working even when some components fail.

Fault tolerance does not mean failures never happen. It means failures are expected and handled.

Why It Matters

At scale, failure is normal:

- Servers crash
- Disks fail
- Network calls timeout
- Databases become overloaded
- Queues get delayed
- Third-party APIs fail

Example

If one backend server fails, a load balancer should stop sending traffic to it and route traffic to healthy servers.

Common Interview Angle

Interviewers may ask what happens if a service, database, or region goes down.

Good answers mention:

- Redundancy
- Retry with backoff
- Timeouts
- Circuit breakers
- Replication
- Failover
- Idempotency
- Monitoring and alerts

3.5 Graceful Degradation

What It Means

Graceful degradation means when part of the system fails, the system continues with reduced functionality instead of failing completely.

Why It Matters

Partial service is often better than total outage.

Example

In an e-commerce app:

- Product pages still load.
- Search still works.
- Recommendations are hidden if the recommendation service fails.
- Checkout may be disabled only if payment service is unavailable.

Common Interview Angle

Interviewers may ask how your system behaves when dependencies fail.

Good answer:

I would identify critical and non-critical features. Critical paths should fail safely. Non-critical features can use cached data, default responses, or be temporarily hidden.

3.6 Replication

What It Means

Replication means copying data across multiple machines.

Why It Matters

Replication improves:

- Read scalability
- Fault tolerance

- Disaster recovery
- Latency when replicas are near users

Example

A primary database handles writes. Read replicas handle read traffic.

Common Interview Angle

Interviewers may ask about replication lag.

Answer:

Read replicas can be behind the primary. For user-critical reads after writes, read from primary or use session consistency.

3.7 Failover

What It Means

Failover means switching traffic from a failed component to a healthy backup.

Why It Matters

Failover reduces downtime.

Example

If the primary database fails, a replica is promoted to primary.

Common Interview Angle

Interviewers may ask:

What happens to writes during failover?

Answer:

Writes may be paused briefly. The system must avoid split-brain, where two primaries accept writes independently.

3.8 Idempotency

What It Means

An idempotent operation can be retried multiple times without changing the result incorrectly.

Why It Matters

Retries are common in fault-tolerant systems. Without idempotency, retries can cause duplicate orders, duplicate payments, or duplicate messages.

Example

Payment request with `idempotency_key = order_123_payment_1`.

If the client retries, the server returns the same payment result instead of charging again.

Common Interview Angle

Interviewers often ask about retry safety in payment or booking systems.

4. Real-World Example

Example: E-Commerce Checkout System

Consider an e-commerce checkout flow:

- 1. User adds item to cart.
- 2. User clicks checkout.
- 3. System verifies inventory.
- 4. System creates order.
- 5. System processes payment.
- 6. System sends confirmation.

Reliability decisions:

Step	Reliability Concern	Design Choice
Check inventory	Avoid overselling	Strong consistency or reservation
Create order	Avoid duplicate orders	Idempotency key
Process payment	Avoid double charge	Idempotent payment API
Send email	Can be delayed	Async queue
Show recommendations	Non-critical	Hide if service fails
Cart page	Should stay available	Cache cart or use fallback

If the recommendation service fails, checkout should still work. If the payment service fails, checkout should fail safely and not create a paid order incorrectly.

This is graceful degradation plus fault tolerance.

5. Diagrams / Mental Models

CAP During Network Partition

Before partition:

Client

|

v
 Replica A <----sync----> Replica B

 Both replicas communicate and agree on data.

 During partition:

 Client 1 ---> Replica A X Replica B <--- Client 2
 network break

 Choice:
 1. Allow both replicas to accept writes: available but may conflict.
 2. Allow only one side to accept writes: consistent but less available.

Reliability Layer Model

```

User Request
  |
  v
Load Balancer
  |
  v
Healthy Backend Servers
  |
  v
Timeouts + Retries + Circuit Breakers
  |
  v
Database / Cache / Queue
  |
  v
Replication + Backup + Monitoring
  
```

Failure Handling Flow

```

Dependency call fails
  |
  v
Was it critical?
  |         |
  | yes     | no
  |         |
  v         v
Fail safely  Use fallback
Return error  cached/default/hidden feature
  
```

Strong vs Eventual Consistency

Strong consistency:

Write -> all required replicas confirm -> read latest data

Eventual consistency:

Write -> primary confirms -> replicas update later -> reads may be stale briefly

6. Common Interview Questions

1. What is consistency in distributed systems?

Consistency means users and services see correct data according to expected rules. In replicated systems, it usually means replicas agree on the latest value.

Key points interviewer expects:

- Correctness of data
- Strong vs eventual consistency
- Replication lag
- Use case based trade-off

Common mistakes:

- Saying consistency only means database constraints
- Ignoring replicas
- Claiming every system needs strong consistency

2. What is availability?

Availability means the system responds to valid requests within acceptable time, even when some components fail.

Key points interviewer expects:

- Non-error response
- Redundancy
- Failover
- Health checks
- Partial failure handling

Common mistakes:

- Confusing availability with low latency
- Assuming available means always latest data
- Ignoring dependency failures

3. Explain CAP theorem.

CAP theorem says that during a network partition, a distributed system must choose between consistency and availability. Partition tolerance is necessary because real networks can fail.

Key points interviewer expects:

- C = consistency
- A = availability
- P = partition tolerance
- Real choice is CP vs AP during partition

Common mistakes:

- Saying a system can choose CA in a real distributed system
- Thinking CAP applies during normal operation only
- Treating CAP as a database ranking system

4. What is the difference between CP and AP systems?

CP systems preserve consistency during partition, possibly rejecting requests. AP systems remain available during partition, possibly serving stale or conflicting data.

Key points interviewer expects:

- CP favors correctness
- AP favors response
- Choice depends on domain

Common mistakes:

- Saying CP systems are always better
- Saying AP systems are incorrect
- Not connecting the choice to business requirements

5. Give examples where strong consistency is needed.

Strong consistency is needed in payments, bank balances, inventory reservation, ticket booking, access control, and order state transitions.

Key points interviewer expects:

- Financial correctness
- Preventing double booking
- Preventing overselling
- Critical state updates

Common mistakes:

- Using strong consistency everywhere
- Forgetting performance and availability cost

6. Give examples where eventual consistency is acceptable.

Eventual consistency is acceptable for likes, views, feeds, notifications, analytics, comments count, search indexing, and recommendations.

Key points interviewer expects:

- Slightly stale data is acceptable
- Better availability and performance
- Background reconciliation

Common mistakes:

- Saying eventual consistency means data loss
- Not explaining convergence

7. What is fault tolerance?

Fault tolerance means a system continues functioning when some components fail.

Key points interviewer expects:

- Redundancy
- Replication
- Failover
- Timeouts
- Retries
- Circuit breakers

Common mistakes:

- Saying backups alone provide fault tolerance
- Ignoring automatic recovery
- Forgetting monitoring

8. What is graceful degradation?

Graceful degradation means the system continues offering core functionality when some non-critical parts fail.

Key points interviewer expects:

- Identify critical vs non-critical features
- Use fallback behavior
- Avoid total outage

Common mistakes:

- Hiding all errors silently
- Letting optional services break critical flows

9. How do retries help reliability?

Retries help recover from temporary failures such as network timeouts or overloaded services.

Key points interviewer expects:

- Use limited retries
- Use exponential backoff

- Use jitter
- Ensure idempotency

Common mistakes:

- Retrying forever
- Retrying non-idempotent operations blindly
- Causing retry storms

10. What is a circuit breaker?

A circuit breaker stops calls to a failing dependency for some time, preventing cascading failures.

Key points interviewer expects:

- Closed state: calls allowed
- Open state: calls blocked
- Half-open state: test recovery
- Protects system from overload

Common mistakes:

- Confusing it with rate limiting
- Not mentioning fallback

11. How do you prevent duplicate payment during retries?

Use an idempotency key. The server stores the result of the first request for that key and returns the same result for retries.

Key points interviewer expects:

- Idempotency key
- Unique request identifier
- Server-side deduplication
- Safe retry behavior

Common mistakes:

- Relying only on client not to retry
- Using timestamp as the only duplicate check

12. What happens if a read replica is stale?

The user may read old data due to replication lag. For critical read-after-write flows, read from primary or use session consistency.

Key points interviewer expects:

- Replication lag
- Read-your-writes problem
- Primary reads for critical paths

Common mistakes:

- Assuming replicas are always up to date
- Ignoring user experience after writes

7. Deep-Dive Questions

1. How would you design checkout to avoid overselling inventory?

Use strong consistency for inventory reservation. When checkout starts, reserve inventory using an atomic transaction or conditional update. The reservation should have an expiry time. If payment fails or times out, release the reservation.

Important points:

- Atomic inventory decrement
- Reservation state
- Expiry cleanup
- Idempotent order creation
- Avoid double booking

2. How do you handle split-brain in database failover?

Split-brain happens when two nodes both think they are primary and accept writes. Prevent it using leader election, quorum, fencing tokens, and strict failover rules.

Important points:

- Only one primary should accept writes
- Quorum prevents isolated minority from becoming primary
- Fencing prevents old primary from writing after demotion

3. How do quorum reads and writes improve consistency?

In a system with N replicas, choose write quorum W and read quorum R . If $R + W > N$, reads overlap with the latest successful write, improving consistency.

Example:

N	W	R	$R + W > N?$
3	2	2	Yes
5	3	3	Yes
5	2	2	No

Trade-off:

- Higher quorum improves consistency.
- Lower quorum improves availability and latency.

4. How do you prevent cascading failures?

Use timeouts, circuit breakers, bulkheads, rate limits, load shedding, fallback responses, and queue-based buffering.

Example:

If recommendation service is slow, backend threads should not wait forever. Use a timeout and return the page without recommendations.

5. How would you make a multi-region system reliable?

Use replication across regions, health checks, traffic routing, regional failover, data backups, and clear consistency rules.

Design choices:

- Active-passive: simpler, one primary region
- Active-active: lower latency, harder conflict resolution
- Synchronous replication: stronger consistency, higher latency
- Asynchronous replication: better latency, possible data loss during disaster

8. Comparison Tables

Consistency vs Availability

Aspect	Consistency	Availability
Main goal	Correct/latest data	System responds
During partition	May reject requests	May return stale data
Best for	Payments, inventory, permissions	Feeds, likes, search, analytics
User impact	Fewer wrong results	Fewer failed requests
Cost	Higher latency or downtime	Possible stale/conflicting data

Strong Consistency vs Eventual Consistency

Aspect	Strong Consistency	Eventual Consistency
Read after write	Always latest	May be stale temporarily
Latency	Higher	Lower
Availability	Lower during failures	Higher during failures
Complexity	Coordination-heavy	Conflict/reconciliation-heavy
Examples	Bank balance, inventory	Likes, feed, view count

CP vs AP Systems

Aspect	CP System	AP System
Full form	Consistency + Partition tolerance	Availability + Partition tolerance
During partition	Rejects some requests	Accepts requests
Data correctness	Prioritized	May be temporarily inconsistent
Availability	Reduced	Preserved
Good for	Critical writes	User-facing read-heavy features

Fault Tolerance vs High Availability

Aspect	Fault Tolerance	High Availability
Meaning	Works despite component failure	Stays accessible most of the time
Focus	Surviving faults	Reducing downtime
Example	Replicated service continues after node crash	99.99% uptime target
Requires	Redundancy and failover	Monitoring, health checks, recovery

Graceful Degradation vs Fail Fast

Aspect	Graceful Degradation	Fail Fast
Meaning	Continue with reduced features	Stop quickly when unsafe
Best for	Non-critical features	Critical correctness paths
Example	Hide recommendations	Reject payment if gateway uncertain
Risk	Users may see limited experience	Users may see errors

Retry vs Circuit Breaker

Aspect	Retry	Circuit Breaker
Purpose	Recover from temporary failure	Stop calling failing service
Works best when	Failure is brief	Dependency is unhealthy
Risk	Retry storm	Overly aggressive blocking
Needs	Backoff and idempotency	Fallback and recovery checks

9. Common Mistakes

- Saying CAP means you can choose only two out of three always.
- Forgetting that partition tolerance is required in real distributed systems.
- Using strong consistency for everything without considering latency and availability.
- Using eventual consistency for payments or inventory without safeguards.
- Retrying requests without idempotency.

- Not setting timeouts for remote calls.
- Letting optional services break critical user flows.
- Confusing fault tolerance with backup.
- Ignoring replication lag.
- Not explaining what happens during failover.
- Assuming cache always improves reliability.
- Forgetting monitoring, alerts, and recovery.

10. Edge Cases / Special Cases

Read-After-Write Consistency

After a user updates data, they expect to see their own update immediately. Even eventually consistent systems may need session consistency for better user experience.

Duplicate Requests

Clients may retry because they did not receive a response. The server may have already processed the request. Use idempotency keys.

Partial Failure

One service may fail while the rest of the application works. Design each dependency with timeout and fallback behavior.

Network Timeout Does Not Mean Failure

If payment API times out, payment may have succeeded. Do not blindly retry without idempotency or reconciliation.

Stale Cache

Cache can improve availability but may serve old data. Use TTL, invalidation, or versioning based on correctness needs.

Split-Brain

During failover, two primaries can cause conflicting writes. Use quorum, leader election, and fencing.

Cascading Failure

One slow dependency can consume threads and bring down the whole service. Use timeouts, circuit breakers, and bulkheads.

Queue Backlog

Queues improve resilience, but if consumers are down, backlog grows. Monitor queue length and processing delay.

11. How to Explain in Interview

Reliability means designing a system to keep working correctly even when parts fail. For critical operations like payments or inventory, I would prefer consistency and fail safely. For non-critical features like feeds, likes, recommendations, or analytics, I can accept eventual consistency and use fallbacks. During network partitions, CAP theorem says we must choose between consistency and availability. A good design uses replication, failover, timeouts, retries with idempotency, circuit breakers, monitoring, and graceful degradation.

12. Quick Revision Notes

Key Definitions

- Consistency: Users see correct data according to system rules.
- Availability: System responds to valid requests.
- CAP theorem: During partition, choose consistency or availability.
- Fault tolerance: System survives component failures.
- Graceful degradation: System keeps core features working when some parts fail.
- Idempotency: Safe to repeat the same operation.
- Failover: Switch from failed component to backup.
- Replication lag: Delay before replicas receive latest data.

Important Points

- Payments need correctness more than availability.
- Feeds and analytics can usually tolerate stale data.
- Retries need backoff and idempotency.
- Cache improves performance but can create stale reads.
- Timeouts are mandatory for remote calls.
- Monitoring is part of reliability, not an optional extra.

Common Comparisons

Comparison	Must Remember
Strong vs eventual consistency	Latest data vs temporary stale data
CP vs AP	Reject during partition vs respond during partition
Retry vs circuit breaker	Try again vs stop calling failing dependency
Fault tolerance vs availability	Survive faults vs reduce downtime
Graceful degradation vs fail fast	Reduced features vs safe rejection

Must-Remember Facts

- CAP is about behavior during network partitions.
- Partition tolerance is unavoidable in distributed systems.
- Availability does not guarantee latest data.
- Eventual consistency means data converges later.
- Network timeout does not prove the operation failed.
- Idempotency is essential for safe retries.

Interview Traps

- Do not say "choose CA" for a real distributed system under partition.
- Do not retry payments without idempotency.
- Do not make optional services part of the critical path.
- Do not ignore stale replicas after writes.
- Do not claim 100% availability.

13. Practice Tasks

Task 1: Classify Consistency Requirements

For each feature, decide strong consistency or eventual consistency:

Feature	Your Choice	Reason
Bank transfer	Strong consistency	Money must not be lost or duplicated
Instagram likes count	Eventual consistency	Slight delay is acceptable
Flight seat booking	Strong consistency	Avoid double booking
Search indexing	Eventual consistency	New data can appear after delay
User password update	Strong consistency	Security-critical

Task 2: Design Failure Behavior

For an e-commerce app, decide what happens when each dependency fails:

Failed Dependency	Expected Behavior
Recommendation service	Hide recommendations
Payment gateway	Stop checkout safely
Email service	Queue email for later
Cache	Read from database
Read replica	Read from primary or another replica

Task 3: Draw a CAP Decision

Draw what your system does during network partition:

Replica A X Replica B

Question:

Do both accept writes?

If yes: AP behavior, conflict handling needed.

If no: CP behavior, some requests rejected.

Task 4: Implement Idempotency in Pseudocode

```
function createPayment(orderId, amount, idempotencyKey):  
    existing = findPaymentByKey(idempotencyKey)  
    if existing exists:  
        return existing.result  
  
    result = chargePaymentGateway(orderId, amount)  
    savePaymentResult(idempotencyKey, result)  
    return result
```

Task 5: Analyze a Retry Policy

Given this policy:

```
Retry every failed request immediately up to 100 times.
```

Problems:

- Can overload the dependency.
- Can create duplicate side effects.
- Can increase latency.
- Can cause cascading failure.

Better policy:

```
Retry 2-3 times with exponential backoff and jitter.  
Only retry idempotent operations.  
Use circuit breaker if failures continue.
```

Task 6: Explain a Real Incident

Pick any outage you know and answer:

- What failed?
- Was it a full failure or partial failure?
- Was there graceful degradation?
- What could have improved reliability?

14. Final Cheat Sheet

Item	Cheat Sheet
Core definition	Reliability means the system works correctly and usefully despite failures
Consistency	Correct/latest data according to rules

Item	Cheat Sheet
Availability	System responds to valid requests
CAP theorem	During partition, choose consistency or availability
Fault tolerance	Survive component failures using redundancy and recovery
Graceful degradation	Keep core features working when optional parts fail
Strong consistency examples	Payments, bank balance, inventory, booking
Eventual consistency examples	Likes, feeds, view counts, search, analytics
Most asked questions	CAP, CP vs AP, strong vs eventual consistency, retries, failover
Common comparisons	Consistency vs availability, CP vs AP, retry vs circuit breaker
Biggest trap	Retrying non-idempotent operations like payments
One-line interview answer	Reliability is about choosing the right consistency and availability trade-offs, handling failures with redundancy and recovery, and degrading non-critical features instead of taking the whole system down.